

LicenseGuard

Software License Management System

CRUD OPERATIONS DOCUMENTATION

Project	LicenseGuard
Database	MySQL
Backend	Spring Boot REST API
Frontend	Java-based desktop frontend / JavaFX
API Base URL	http://localhost:8080
Database Tables	7
CRUD Scope	Departments, Users, Vendors, Software, Licenses, Assignments, Renewals

Architecture

Frontend
↓
Java / JavaFX
↓
REST API
↓
Spring Boot
↓
MySQL

Purpose: This document records the Create, Read, Update and Delete operations implemented for the LicenseGuard application, together with the REST endpoints, database mapping and special license-management operations.

Source basis: the existing LicenseGuard Database Design & ERD document defines the seven tables, attributes, relationships and SQL operations. The CRUD/API sections below reflect the verified Spring Boot backend implementation described in the project work.

1. CRUD Overview

CRUD stands for **Create, Read, Update and Delete**. In LicenseGuard, CRUD is implemented through REST APIs in the Spring Boot backend. The backend uses Spring Data JPA to communicate with MySQL. The frontend communicates with the backend using HTTP/JSON and does not access MySQL directly.

Entity / Module	Create	Read	Update	Delete / Special Operation
Department	POST	GET	PUT	DELETE
User	POST	GET	PUT	DELETE / deactivate
Vendor	POST	GET	PUT	DELETE
Software	POST	GET	PUT	DELETE
License	POST	GET	PUT	DELETE
License Assignment	POST	GET	Unassign	DELETE
Renewal	POST	GET	—	DELETE

The database design contains these seven tables: departments, users, vendors, software, licenses, license_assignments and renewals. The original database document defines the primary/foreign keys and relationships that support these operations. (Database design document, pages 2–7.)

2. REST API Convention

All successful API responses use the application's standard response wrapper containing success, message, data and timestamp. Validation and business-rule failures are returned as structured error responses.

HTTP Method	CRUD Meaning	Typical Use
POST	Create	Create a new record
GET	Read	Retrieve one or more records
PUT	Update	Modify an existing record or perform a supported state change
DELETE	Delete	Remove a record when deletion is allowed

The verified backend also returns appropriate HTTP codes such as 200 OK, 201 Created, 400 Bad Request, 404 Not Found and 409 Conflict.

3. Department CRUD Operations

The **departments** table stores company departments. Its primary key is **department_id**. The database document defines department_name as required and description as optional.

Operation	REST API	Purpose
Create	POST /api/departments	Create a department
Read all	GET /api/departments	List all departments
Read one	GET /api/departments/{id}	Retrieve one department
Update	PUT /api/departments/{id}	Modify department details
Delete	DELETE /api/departments/{id}	Delete a department where allowed

Database-level SQL examples from the project:

```
-- Read
SELECT * FROM departments;

-- Create
INSERT INTO departments (department_name, description)
VALUES ('IT', 'Information Technology Department');

-- Update
UPDATE departments
SET department_name = 'Information Technology',
    description = 'IT Department'
WHERE department_id = 1;

-- Delete
DELETE FROM departments
WHERE department_id = 1;
```

Relationship: DEPARTMENT 1 : N USER. A department can contain many users; each user belongs to one department.

4. User CRUD Operations

The **users** table stores employees/system users who may receive licenses. It contains department_id as a foreign key and email as a unique field.

Operation	REST API	Purpose
Create	POST /api/users	Create a user
Read all	GET /api/users	List users
Read filtered	GET /api/users?departmentId={id}	List users by department
Read one	GET /api/users/{id}	Retrieve one user
Update	PUT /api/users/{id}	Modify user details
Delete	DELETE /api/users/{id}	Delete where permitted

Important implementation behavior:

User passwords are accepted for writes but are not returned in API response DTOs. The verified backend uses a write-only JSON property/DTO approach so the password is excluded from normal responses.

```
-- Read all users
SELECT * FROM users;

-- Update user email
UPDATE users
SET email = 'rahul.sharma@licenseguard.com'
WHERE user_id = 2;

-- Preferred deactivation
UPDATE users
SET status = 'INACTIVE'
WHERE user_id = 3;
```

Relationship: DEPARTMENT 1 : N USER. Users are also connected to license assignments and renewals.

5. Vendor CRUD Operations

The **vendors** table stores software vendors or publishers. The main fields are vendor_id, vendor_name, contact_email and website.

Operation	REST API	Purpose
Create	POST /api/vendors	Create vendor
Read all	GET /api/vendors	List vendors
Read one	GET /api/vendors/{id}	Retrieve vendor
Update	PUT /api/vendors/{id}	Modify vendor
Delete	DELETE /api/vendors/{id}	Delete vendor where allowed

```
-- Read
SELECT * FROM vendors;

-- Create
INSERT INTO vendors (vendor_name, contact_email, website)
VALUES ('Microsoft', 'support@microsoft.com',
        'https://www.microsoft.com');

-- Update
UPDATE vendors
SET contact_email = 'new-support@example.com'
WHERE vendor_id = 1;
```

Relationship: VENDOR 1 : N SOFTWARE. One vendor can provide many software products.

6. Software CRUD Operations

The **software** table stores software products and versions. `vendor_id` links each software product to a vendor.

Operation	REST API	Purpose
Create	POST /api/software	Create software
Read all	GET /api/software	List software
Read filtered	GET /api/software?vendorId={id}	List software for a vendor
Read one	GET /api/software/{id}	Retrieve software
Update	PUT /api/software/{id}	Modify software
Delete	DELETE /api/software/{id}	Delete software where allowed

```
-- Read all
SELECT * FROM software;

-- Read with vendor
SELECT *
FROM software
WHERE vendor_id = 1;

-- Create
INSERT INTO software
(software_name, version, vendor_id, description)
VALUES ('Microsoft 365', '2026', 1,
       'Productivity and office software suite');

-- Update
UPDATE software
SET version = '2026.1'
WHERE software_id = 1;
```

Relationship: VENDOR 1 : N SOFTWARE and SOFTWARE 1 : N LICENSE.

7. License CRUD Operations

The **licenses** table is the core license inventory. It contains the license key, type, dates, total seats, available seats, cost and status.

Operation	REST API	Purpose
Create	POST /api/licenses	Create license
Read all	GET /api/licenses	List licenses
Read by status	GET /api/licenses?status={status}	Filter by status
Read by software	GET /api/licenses?softwareId={id}	Filter by software
Read one	GET /api/licenses/{id}	Retrieve license
Update	PUT /api/licenses/{id}	Modify license
Delete	DELETE /api/licenses/{id}	Delete where allowed
Expiring	GET /api/licenses/expiring?days=30	Find licenses expiring soon

```
-- Read active licenses
SELECT license_id, license_key, license_type,
       expiry_date, status
FROM licenses
WHERE status = 'ACTIVE';

-- Find licenses expiring within 30 days
SELECT license_id, license_key, expiry_date
FROM licenses
WHERE expiry_date <= DATE_ADD(CURDATE(), INTERVAL 30 DAY)
AND status <> 'EXPIRED';
```

Seat management: available_seats is maintained by backend business logic. Assignment decreases available seats; unassignment restores a seat without exceeding total_seats.

8. License Assignment Operations

The **license_assignments** table records which user receives which license and preserves assignment history through `assigned_date`, `unassigned_date` and `status`.

Operation	REST API	Purpose
Create / Assign	POST /api/license-assignments	Assign a license to a user
Read all	GET /api/license-assignments	List assignments
Read by license	GET /api/license-assignments?licenseId={id}	Filter by license
Read by user	GET /api/license-assignments?userId={id}	Filter by user
Unassign / Update	PUT /api/license-assignments/{id}/unassign	Make assignment inactive and restore seat
Delete	DELETE /api/license-assignments/{id}	Delete assignment where allowed

Business rules verified in the project:

Rule	Result
<code>available_seats > 0</code>	Assignment is allowed and <code>available_seats</code> decreases.
<code>available_seats = 0</code>	Request is rejected with 400 Bad Request / <code>InsufficientSeatsException</code> .
Same user + same active license	Duplicate active assignment is rejected with 409 Conflict.
Unassign active assignment	Status becomes <code>INACTIVE</code> , <code>unassigned_date</code> is set and a seat is restored.
Unassign inactive assignment	Request is rejected with 400 Bad Request.

This module is more than basic CRUD because it contains the application's seat-management business rules.

9. Renewal Operations

The **renewals** table stores license renewal history. A renewal captures the old expiry date, new expiry date, renewal date, optional cost, renewed_by user and remarks.

Operation	REST API	Purpose
Create / Renew	POST /api/renewals	Renew a license and create renewal history
Read all	GET /api/renewals	List renewal history
Read by license	GET /api/renewals?licenseId={id}	List renewals for a license
Read one	GET /api/renewals/{id}	Retrieve renewal record
Delete	DELETE /api/renewals/{id}	Delete renewal where allowed

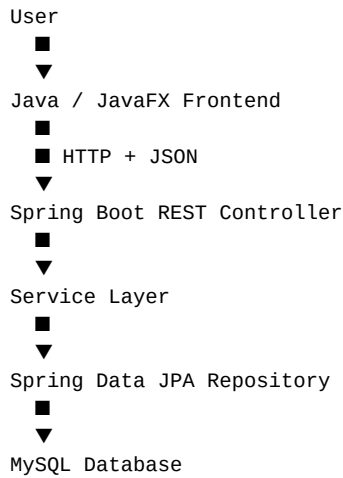
Renewal business rules verified in the project:

Step	Backend behavior
1	Read the current license expiryDate.
2	Store the current expiry as oldExpiryDate.
3	Require newExpiryDate to be later than the current expiry.
4	Update the license expiryDate to newExpiryDate.
5	Set the license status to ACTIVE.
6	Create a Renewal history record.
7	Complete the operation transactionally.

Invalid renewal dates are rejected with 400 Bad Request. This protects the license lifecycle and prevents a renewal from shortening or leaving the expiry unchanged.

10. CRUD Data Flow

The completed architecture keeps the Java-based frontend separate from database persistence. CRUD requests flow through the REST API and Spring Boot backend.



Layer	Responsibility
JavaFX Frontend	Forms, tables, validation hints, navigation and displaying results.
REST Controller	Receives HTTP requests and returns HTTP/JSON responses.
Service	Implements business rules and transactions.
Repository	Performs persistence operations through JPA/Hibernate.
MySQL	Stores the seven LicenseGuard tables and their relationships.

Important: The frontend must not use JDBC or a MySQL connection. It communicates only with the Spring Boot REST API.

11. CRUD Validation and Error Handling

HTTP	Meaning in LicenseGuard
200 OK	Successful GET/PUT operation
201 Created	Successful POST operation
400 Bad Request	Invalid request, validation failure, invalid business operation or insufficient seats
404 Not Found	Requested entity does not exist
409 Conflict	Duplicate resource such as an existing email, license key or vendor name
500 Internal Server Error	Unexpected server-side error

The backend uses structured exception handling for `ResourceNotFoundException`, `BadRequestException`, `DuplicateResourceException` and `InsufficientSeatsException`. Validation errors are returned with field-level details.

12. SQL CRUD Reference

The existing database document explicitly demonstrates SELECT, UPDATE and DELETE operations. CREATE is represented by INSERT statements and table-creation statements. The following reference consolidates the CRUD concepts for the project.

CREATE

```
INSERT INTO departments (department_name, description)
VALUES ('IT', 'Information Technology Department');

INSERT INTO vendors (vendor_name, contact_email, website)
VALUES ('Microsoft', 'support@microsoft.com',
        'https://www.microsoft.com');

INSERT INTO software
(software_name, version, vendor_id, description)
VALUES ('Microsoft 365', '2026', 1,
        'Productivity and office software suite');
```

READ

```
SELECT * FROM departments;
SELECT * FROM users;
SELECT * FROM vendors;
SELECT * FROM software;
SELECT * FROM licenses;

-- Assignment details
SELECT la.assignment_id,
       u.name AS user_name,
       s.software_name,
       l.license_key,
       la.assigned_date,
       la.status
FROM license_assignments la
JOIN users u ON la.user_id = u.user_id
JOIN licenses l ON la.license_id = l.license_id
JOIN software s ON l.software_id = s.software_id;
```

UPDATE

```
UPDATE users
SET email = 'rahul.sharma@licenseguard.com'
WHERE user_id = 2;

UPDATE licenses
SET status = 'SUSPENDED'
WHERE license_id = 4;

UPDATE license_assignments
SET status = 'INACTIVE',
    unassigned_date = CURDATE()
WHERE assignment_id = 1;
```

DELETE / DEACTIVATE

```
-- Preferred for business records:
UPDATE users
SET status = 'INACTIVE'
WHERE user_id = 3;

-- Physical deletion only when appropriate:
DELETE FROM users
WHERE user_id = 3;
```

The database document recommends status-based updates instead of physical deletion for important license-management records because audit history may be required. (Database design document, page 14.)

13. CRUD Implementation Verification

The backend CRUD implementation was verified through Maven build/test and endpoint testing. The reported verification confirms that the REST operations are connected to MySQL through JPA/Hibernate.

Verification	Result
Spring Boot application startup	PASS
MySQL connection	PASS — localhost:3306/licenseguard
JPA/Hibernate initialization	PASS
Departments CRUD endpoints	PASS
Users CRUD endpoints	PASS
Vendors CRUD endpoints	PASS
Software CRUD endpoints	PASS
Licenses CRUD + expiring endpoint	PASS
License assignment + seat management	PASS
Renewal creation + expiry update	PASS
Validation/error handling	PASS
mvn clean test	BUILD SUCCESS — 1 test, 0 failures

The verified backend compiled 58 source files with no compilation errors. The application started on port 8080 and connected successfully to MySQL database **licenseguard**. Schema validation used **ddl-auto=validate** and reported no schema mismatches.

14. Database Relationship Support

Relationship	Cardinality	CRUD Impact
DEPARTMENT → USER	1 : N	Users can be created/read with department association.
VENDOR → SOFTWARE	1 : N	Software records reference vendors.
SOFTWARE → LICENSE	1 : N	Licenses reference software products.
USER → LICENSE_ASSIGNMENT	1 : N	Assignments reference users.
LICENSE → LICENSE_ASSIGNMENT	1 : N	Assignments reference licenses and manage seats.
LICENSE → RENEWAL	1 : N	Renewals preserve license history.
USER → RENEWAL	1 : N	Renewals record the user who performed them.

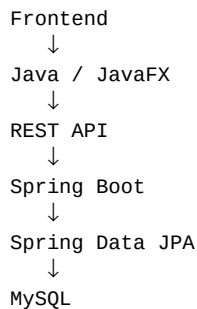
These relationships match the ERD and table structure in the supplied database-design PDF, especially the relationship summary on pages 7 and 15–16.

15. Final CRUD Summary

LicenseGuard implements a complete REST-based CRUD foundation for its main master-data entities and adds domain-specific operations for license assignment and renewal management.

Module	CRUD Status	Key Feature
Departments	Complete	Department master-data management
Users	Complete	Department filtering and password exclusion from responses
Vendors	Complete	Vendor master-data management
Software	Complete	Vendor relationship and filtering
Licenses	Complete	Status, expiry and seat management
Assignments	CRUD + business operations	Seat decrement/increment and duplicate protection
Renewals	Create/Read/Delete + business operations	Expiry extension and renewal history

Final architecture



Conclusion: The CRUD operations are implemented at the application level through Spring Boot REST APIs, with MySQL providing persistent storage. The Java-based frontend is designed to consume these APIs rather than connect directly to the database.

Reference: LicenseGuard — Database Design, Entity-Relationship Diagram (ERD) & SQL Implementation, 16-page project document supplied with this task.